

CAPÍTULO IV: FASE DE EJECUCIÓN

En las metodologías de desarrollo de software implícitamente se incorporan procesos de integración de recursos disponibles para el proyecto a fin de llevar a cabo la planificación desarrollada. Esto se puede definir como una fase de ejecución. Modelo Móvil Sprint implementa buenas practicas a seguir para efectuar la correcta incorporación de los instrumentos, funciones y recursos utilizables.

En la fase anterior de diseño se establecieron los prototipos desarrollados en base a las consideraciones que fueron planteadas tras el análisis de los requerimientos del proyecto de software.

A continuación, se procede a realizar la implementación de aquellos prototipos en función de las sugerencias establecidas para cada parte del proceso que involucra las etapas de codificación, el diseño de pruebas y la refactorización.

4.1 Objetivos

En todo proyecto de software se considera que la etapa en donde se consumen la mayor cantidad de los recursos es en la fase de ejecución puesto que lleva a cabo toda la planificación y construcción de los análisis realizados en cuanto a los requerimientos que se desean satisfacer.

Al ser una etapa donde se consumen gran parte de los recursos se debe gestionar de manera óptima para evitar contratiempos por falta de los mismos. Por ello, el objetivo principal de esta etapa es la implementación de buenas prácticas orientadas a la simplificación del trabajo y la maximización de la producción, además de integrar herramientas que permitan obtener un resultado satisfactorio.

4.2 Herramientas involucradas

Modelo Móvil Sprint emplea tres herramientas específicas para la fase de ejecución, las cuales le permiten a los grupos de desarrollo mantener una organización limpia y fluida indistintamente si se trata de macro proyectos o micro proyectos.

Las herramientas seleccionadas para implementar durante esta fase se clasifican en:

- Control de versiones.
- Editor de código.

Para el control de versiones se aplican dos herramientas las cuales son: Github y Azure DevOps. Aunque su función es similar, cabe resaltar que en conjunto se convierten en un sistema bastante completo para el control de versiones. Esto se debe a la compatibilidad que existe entre sí, mientras Azure DevOps mantiene un enfoque sobre la gestión de las versiones, Github trabaja con repositorios de código.



Ilustración 10. Relación entre Github y Azure DevOps.

Fuente: elaboración propia.

4.3 Actividades en la fase de ejecución

La fase de ejecución está compuesta de manera general por el proceso de codificación y refactorización, donde cada una ejemplifica las buenas prácticas que se recomiendan dentro de la presente metodología. Gran parte de los desarrolladores experimentados emplean y desarrollan distintas buenas prácticas para mantener una codificación más limpia y sobre todo realizar un mayor control sobre el desarrollo de funciones, variables, librerías, recursos, entre otras.

De acuerdo con la Ilustración 12, las buenas prácticas planteadas se ejecutan en el siguiente orden con el objetivo de preparar el entorno de trabajo antes de llevarlo a cabo.

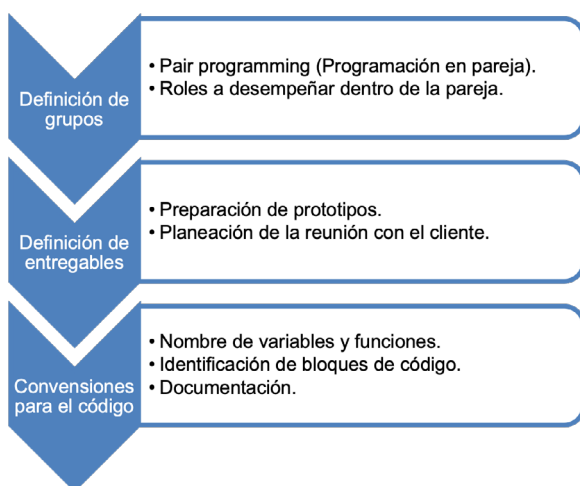


Ilustración 11. Actividades y buenas prácticas en la codificación.

Fuente: elaboración propia.

Mientras que para la refactorización se explican en la Ilustración 13, cabe resaltar que el propósito de la refactorización es simplificar y optimizar los bloques de código para obtener una mayor legibilidad del código, así como un mejor entendimiento. Por ello se enfatiza a plantear las siguientes buenas prácticas:

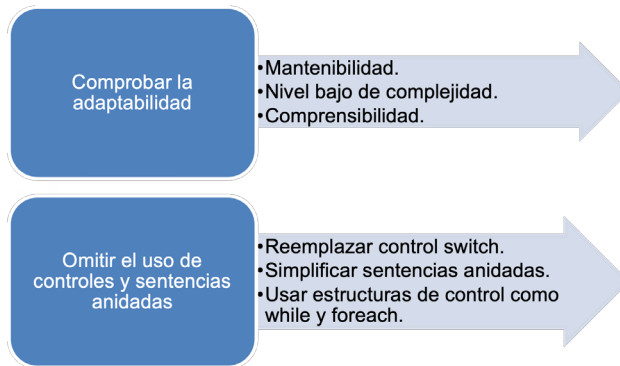


Ilustración 12. Buenas prácticas en el proceso de refactorización.

Fuente: elaboración propia.

4.4 Codificación

La codificación del software según Palomo (2020), lo define como el proceso de llevar los requerimientos funcionales del software al desarrollo del código fuente en un determinado lenguaje de programación. Generalmente esta disciplina suele ser desempeñada de forma desordenada e ineficiente por la carencia de herramientas y/o buenas prácticas que garanticen la compresión del código por desarrolladores externos.

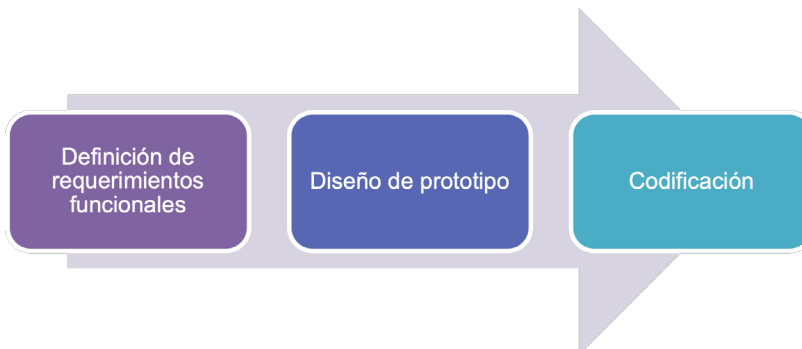


Ilustración 13. Proceso del desarrollo del prototipo.

Fuente: elaboración propia.

La presente metodología se enfoca en cuatro aspectos importantes como la fomentación del trabajo en equipo, el desarrollo de muestras o entregables, el uso de estándares para la codificación y la documentación.

4.4.1 Definición de grupos

La idea de conformar grupos de desarrolladores surgió por la necesidad de optimizar recursos como el tiempo puesto que antes de su llegada, las metodologías de desarrollo tradicionales ocupaban más tiempo en llevar a cabo la elaboración del código y revisión del mismo.

Por lo general en las metodologías de desarrollo con enfoque ágil tienen preestablecido la conformación de parejas para llevar a cabo el despliegue de los módulos a construirse.

Modelo Móvil Sprint implementa el concepto de Pair Programming, que necesita de dos programadores se integren al trabajo de desarrollo en un lugar de trabajo. Los integrantes realizan operaciones distintas a lo largo del desarrollo, por ejemplo, cuando una unidad prueba el código, otro miembro considera las clases que cumplen con los requisitos de la prueba. El integrante que codifica o desarrolla recibe el nombre de controlador y el integrante al que se dirige se llama navegador. De forma general, es recomendable que los roles se intercambien al menos cada media hora o después de completar el testeo del módulo desarrollado.

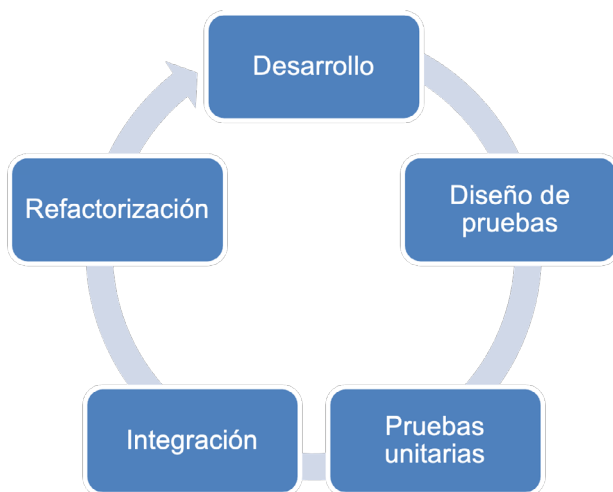


Ilustración 14. Actividades rotativas de la programación en pareja.

Fuente: elaboración propia.

4.4.2 Definición de entregables

Al final de cada interacción de la metodología se realiza la entrega de resultados provistos de los prototipos desarrollados en la fase anterior. Estos resultados que reciben el nombre de entregables deben ser evaluados antes de previamente a la reunión con el cliente.

Es recomendable seleccionar aquellos productos que se encuentren completos o lo más cercanos a satisfacer los requerimientos que fueron utilizados para desarrollar el prototipo.

4.4.3 Convenciones para el código

Los desarrolladores de software generalmente no hacen uso de algún tipo de sistematización a la hora de escribir códigos, por lo que eso puede ser un proceso altamente peligroso porque se expone a problemas como la dificultad de realizar actualizaciones debido a la espesa cantidad de bloques de código, tanto como el nombre de variables, funciones, procesos, librerías entre otras.

En esta metodología se plantea el uso de buenas prácticas a la hora de estructurar los bloques de código, el uso de nombres adecuados para las variables y funciones, la indentación o tipo de sangría.

4.4.3.1 Nombre de variables y funciones

Los nombres de variables deben caracterizarse por ser determinantes, legibles y diferentes a los que usan las librerías con las que se encuentra trabajando. De esa manera fácilmente se evitan colisiones con otros nombres que son propios de frameworks y/o librerías.

Para determinar el nombre de las variables se plantea la siguiente sistematización:

Tabla 20. Buenas prácticas para variables y funciones.

	RECOMENDACIÓN	EJEMPLO
Numero de caracteres máximo	20	<i>String nombre;</i>
Signo de agrupación de nombre en caso de variables	" _ "	<i>String nombre_cliente;</i>

Agrupación de nombres para funciones	A partir de la segunda palabra, la primera letra debe ser en mayúscula	<i>guardarCliente();</i>
Variables recomendadas para las iteraciones en las estructuras iterativas	“i”, “j”, “k” y “n”	<i>for(i=0;i<5;++i){}</i>
Para las funciones usadas constantemente	Se debe escribir todo en mayúsculas, en caso de usar más de una palabra en su descripción se debe usar el signo “_” para la separación.	<i>REFRESCAR_PAGINA();</i>

Fuente: elaboración propia.

4.4.3.2 Identación

Los lenguajes de programación como java, C, C++, entre otros, utilizan signos como el “;” para indicar al depurador hasta dónde llega una línea de código. Sin embargo, actualmente con el auge del desarrollo de aplicaciones móviles y web se emplean lenguajes de programación como Python que tiene un repertorio muy extenso de librerías y funciones para procesar datos y realizar operaciones complejas. En este caso, Python es un lenguaje de programación que no emplea el “;” para indicar el final de una línea de código. En este caso utiliza lo que se conoce como indentación.

Usar correctamente la sangría posibilita que los bloques de código extensos puedan ser legibles estructuralmente sin importar el lenguaje que se encuentre utilizando. Existen muchos beneficios al usar pestañas de 8 espacios para la sangría. La indentación está claramente marcada para facilitar la lectura del código. Al obligar a la función a dividirse en bloques más modulares y bien definidos, también ayuda a mantener limpio el código; si la sangría excede el margen correcto, significa que la función está mal diseñada y debe dividirse para hacerla más modular u otro.

Las pestañas con sangría de 8 espacios también ayudan a que las funciones de diseño quepan en la pantalla, lo que significa que las personas pueden comprender el código sin tener que desplazarse hacia adelante y hacia atrás para comprender el código. A continuación, se plantean unos ejemplos de recomendaciones sobre la indentación.

Tabla 21. Recomendaciones para la indentación de código.

RECOMENDADO	NO RECOMENDADO
<pre>while(\$fila=pg_fetch_array(\$resultado)){ \$pdf->Cell(50,5,'Nombres',0,0,'L') ; \$pdf->Cell(30,5,'Apellidos',0,0,'C') ; \$pdf->Cell(20,5,'Direccion', 2),0,0,'C') ; \$pdf->Cell(20,5,'Ciudad', 2),0,0,'C') ; \$pdf->Cell(20,5,'Telefono',2),0,1,'R') ; }</pre>	<pre>while(\$fila=pg_fetch_array(\$resultado)){ \$pdf->Cell(50,5,'Nombres',0,0,'L') ; \$pdf->Cell(30,5,'Apellidos',0,0,'C') ; \$pdf->Cell(20,5,'Direccion', 2),0,0,'C') ; \$pdf->Cell(20,5,'Ciudad', 2),0,0,'C') ; \$pdf->Cell(20,5,'Telefono',2),0,1,'R') ; }</pre>
<pre>while(\$fila=pg_fetch_array(\$resultado)){ if(\$existe==0){ \$this->Cell(140,5) ; \$this->Cell(120,5) ; \$this->Cell(20,5) ; \$this->Cell(120,5) ; \$this->Cell(20,5) ; \$this->Cell(120,5) ; \$this->Cell(20,5) ; \$existe=1; }else{ \$existe=1; } }</pre>	<pre>while(\$fila=pg_fetch_array(\$resultado)){ if(\$existe==0){ \$this->Cell(140,5) ; \$this->Cell(120,5) ; \$this->Cell(20,5) ; \$this->Cell(120,5) ; \$this->Cell(20,5) ; \$this->Cell(120,5) ; \$this->Cell(20,5) ; \$this->Cell(120,5) ; \$existe=1; }else{ \$existe=1; } }</pre>

Fuente: elaboración propia.

4.4.4 Documentación

Para la documentación se pretende hacer una serie de sugerencias al momento de escribir el programa, el uso de estas consideraciones puede ayudar a la claridad del código desarrollado. Cada programador tiene sus propios estándares para los mejores puntos para colocar la documentación en el código fuente y la mejor manera de implementar la documentación.

El estilo correcto del documento depende del lenguaje de programación y las prácticas de la entidad para la que se va a desarrollar el software y quién lo mantiene. Para realizar la documentación del código se consideran los atributos descritos en la siguiente tabla.

Tabla 22. Parámetros para describir en la documentación.

PARÁMETRO	DESCRIPCIÓN
Entradas	Listado de variables y su definición dentro de la función.
Salidas	Resultado que se obtiene de funciones.

Valor de retorno	Define los valores que son retornados por la función y que significan.
Función	Explique para que sirve dicha función.
Variables	Define el tipo y nombre de variables que son utilizadas en distintos contextos.
Fecha	Indica la fecha en la que se realiza la observación del código
Autor	Indica al responsable del comentario
Rutinas anexas	Describe su relación con otras rutinas (en caso de ser un caso de funciones anidadas)

Fuente: elaboración propia.

A continuación, se realiza un ejemplo en donde se identifica cada uno de los parámetros:

```
/*
 * Entradas: ingresan los campos que pertenecen al detalle de factura
 * Salida: añadir valores a los vectores que vamos a enviar
 * Valor de retorno: ningún valor retorna
 * Función: sirve para extraer valores de elementos DOM y añadir a los arrays
 * Variables: codigo, descripcion, cantidad, iva, precio, total, ivadet
 * Fecha: 15-12-2020
 * Autor: Henry Pardo
 * Rutinas anexas: addTabla()
 */
function addAProd(codigo, descripcion, cantidad, iva, precio, total, ivadet){
    codigo = codigo.toString();
    index = codigoA.indexOf(codigo);
    if(index >= 0){ ...
    }
    else{ ...
    }
}

$('#txtCodA').val(codigoA);
$('#txtDescA').val(descripcionA);
$('#txtCantA').val(cantidadA);
$('#txtIvaA').val(ivaA);
$('#txtPrecA').val(precioA);
$('#txtTotalA').val(totalA);
$('#txtIvaDetalleA').val(ivaDetalleA);

addTabla(codigoA, descripcionA, cantidadA, ivaA, precioA, totalA, ivaDetalleA);
```

Ilustración 15. Ejemplo de documentación.

Fuente: elaboración propia.

4.5 Refactorización

La producción de software está ligada a un proceso estructurado de desarrollo donde se analizan diferentes aspectos para la elaboración de productos con altos índices

de calidad. Los resultados de ejecutar protocolos durante el ciclo de desarrollo de software se ven cristalizados cuando el programa o sistema elaborado es puesto a prueba en la etapa de testeo y satisface a los requerimientos funcionales.

La refactorización consiste en optimizar los bloques de código a partir de su estructura sin que se comprometa su funcionamiento. Modelo Móvil Sprint se enfoca en evitar errores comunes durante la etapa de producción del código. Entre los problemas más comunes esta la duplicidad del código, métodos extensos, el exceso de parámetros que necesitan ciertas funciones, etc. Por ello se plantean dos buenas prácticas para la implementación de la refactorización:

4.5.1 Comprobar la adaptabilidad

Si la estructura de la aplicación no es suficiente para introducir los cambios necesarios, primero refactorice el código para que los cambios sean fáciles de introducir y luego agregue nuevas funciones.

Esta consideración pretende prever el surgimiento de errores durante la etapa de mejoramiento del código, puesto que empezar a modificar sin antes evaluar su adaptabilidad podría generar retrasos en el desarrollo de los entregables.

Sin embargo, al final de aplicar este consejo se obtendrán resultados favorables en aspectos como:

- **Mantenibilidad:** Cuando se desee realizar un mantenimiento sobre el sistema, este será sencillo de realizarlo por la simplicidad de un código limpio y navegable.
- **Nivel bajo de complejidad:** Los niveles de complejidad para resolver el problema o necesidad es relativamente sencillo y fácil de realizar optimizaciones.
- **Comprensibilidad:** El código fuente es más fácil de entender.

4.5.2 Omitir el uso de controles y sentencias anidadas

Evitar la implementación de sentencias anidadas y controles que generan la extensión innecesaria del código. Se debe buscar alternativas que permitan reducir las líneas de código como es el caso de la sentencia de control switch. Utilizar este tipo de sentencias obliga a plantear un menú extenso y difícil de reducir, para ello se puede reemplazar por otras estructuras o funciones que desempeñen las mismas actividades.

```
function getPokemon(type) {  
  let pokemon;  
  switch (type) {  
    case 'Water':  
      pokemon = 'Squirtle';  
      break;  
    case 'Fire':  
      pokemon = 'Charmander';  
      break;  
    case 'Plant':  
      pokemon = 'Bulbasaur';  
      break;  
    case 'Electric':  
      pokemon = 'Pikachu';  
      break;  
    default:  
      pokemon = 'Mew';  
  }  
  return pokemon;  
}
```

Ilustración 16. Ejemplo de función switch.

Fuente: elaboración propia.

```
const pokemon = new Map([  
  ['Water', 'Squirtle'],  
  ['Fire', 'Charmander'],  
  ['Plant', 'Bulbasaur'],  
  ['Electric', 'Pikachu']  
]);  
  
function getPokemon(type) {  
  return pokemon.get(type) || 'Mew';  
}  
  
console.log(getPokemon('Fire'));  
console.log(getPokemon('unknown'));
```

Ilustración 17. Optimización de estructura de control.

Fuente: elaboración propia.